



**Università
di Brescia**

DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

Corso di Laurea Magistrale

in Ingegneria Informatica

Relazione di Progetto del Corso

Algoritmi e Strutture Dati

Docente: Prof.ssa Marina Zanella

Studenti:

Davide Leone - 723335

Filippo Camossi - 734170

Anno Accademico 2024/2025

Indice

1	Introduzione	1
1.1	Strumenti e scelte tecnologiche	1
1.2	Rappresentazione dello stato della cella	1
2	Compito uno: generazione dell'ambiente	3
2.1	Personalizzazione della griglia tramite parametri	3
2.2	Tipologie di ostacoli e modellizzazione logica	3
2.3	Algoritmo di generazione	4
3	Compito due: raggiungibilità e distanza libera	6
3.1	Distanza libera (d_{lib}) e formulazione matematica	6
3.2	Contesto e complemento	6
3.3	Proiezione di raggi geometrici ($O(\max(R, C)^2)$)	7
3.4	Frontiera della chiusura	9
4	Compito tre: ricerca del cammino minimo	11
4.1	Definizione ricorsiva e flusso esecutivo	11
4.2	Analisi della potatura: riga 16 contro riga 17	11
4.3	Potatura branch-and-bound globale	12
4.4	Ottimizzazioni fisiche e implementative	13
4.5	Requisiti funzionali per gruppi di 3 persone	13
5	Compito quattro: sperimentazione e analisi	15
5.1	Analisi di complessità spaziale asintotica	15
5.2	Analisi di complessità temporale asintotica	15
5.3	Nota metodologica sulla misura	16
5.4	Analisi dei risultati sperimentali e discussione dei grafici	16
5.4.1	1. Tempo in funzione della densità di ostacoli	17
5.4.2	2. Confronto delle configurazioni (potatura e ordinamento) . .	17
5.4.3	3. Crescita temporale asintotica (scala bilogaritmica)	18

5.4.4	4. Correlazione tra tempo e celle di frontiera	19
5.4.5	5. Correlazione tra tempo e invocazioni ricorsive	19
5.4.6	6. Verifica formale della simmetria	19
6	Conclusioni	20
6.1	Limitazioni riscontrate	20
6.2	Motivazioni delle scelte implementative	21
6.3	Sviluppi futuri	21
7	Uso del codice	23
7.1	Configurazione e requisiti di sistema	23
7.2	Configurazione del file JSON delle griglie	23
7.3	Guida all'uso della riga di comando	24
7.3.1	1. Generazione di griglie (Compito 1)	24
7.3.2	2. Calcolo di contesto, complemento e frontiera (Compito 2) .	24
7.3.3	3. Calcolo della distanza ideale d_{lib}	25
7.3.4	4. Calcolo del cammino minimo (Compito 3)	25
7.3.5	5. Campagna sperimentale completa (Compito 4)	25
7.3.6	6. Verifica di simmetria formale	25
7.4	Esecuzione della batteria di prove unitarie	26

1 Introduzione

La presente relazione descrive la progettazione, l'implementazione e la campagna sperimentale condotta per il sistema di navigazione e calcolo del cammino minimo su griglia bidimensionale, sviluppato in conformità con i requisiti del corso di Algoritmi e Strutture Dati per l'anno accademico 2024/25.

L'elaborato affronta il calcolo del cammino minimo su griglie bidimensionali in presenza di ostacoli. Il sistema mira a limitare il sovraccarico computazionale tipico delle esplorazioni ricorsive ricorrendo a strutture dati specifiche e a regole di potatura spaziale.

1.1 Strumenti e scelte tecnologiche

Il software è scritto in **Python 3.13** e utilizza:

- **NumPy**: gestisce la rappresentazione della griglia attraverso array multidimensionali, sfruttando le operazioni vettoriali per ottimizzare le letture e le modifiche dello stato.
- **Matplotlib**: genera le visualizzazioni grafiche dei percorsi e produce i grafici per l'analisi delle prestazioni (Compito 4).
- **Argparse**: fornisce l'interfaccia a riga di comando.

Rispetto all'approccio a oggetti, che associa a ogni cella un'istanza dedicata e grava sul gestore della memoria, la rappresentazione tramite array numerici lavora su dati contigui in memoria.

1.2 Rappresentazione dello stato della cella

Ogni cella della griglia è codificata con un numero intero all'interno di una matrice NumPy bidimensionale di tipo `uint8` (1 byte per cella), in base allo schema seguente:

Tabella 1: Rappresentazione dello stato della cella a livello logico e fisico

Stato cella	Valore fisico	Significato operativo
Libera	0	Cella attraversabile (navigabile).
Ostacolo fisso	1	Ostacolo permanente generato nel Compito 1 o importato da JSON.
Ostacolo temporaneo	≥ 2	Cella marcata direttamente in memoria durante il ritracciamento ricorsivo con un identificativo corrispondente a <i>profondità + 2</i> .

Questa codifica numerica consente il **ritracciamento (backtracking) sul posto**. A ogni chiamata ricorsiva di **CAMMINOMIN**, invece di clonare la matrice della griglia (operazione che esaurirebbe la memoria sulle griglie più ampie), il codice sovrascrive il valore intero della cella visitata direttamente nella struttura dati condivisa.

Durante la discesa, i nodi attraversati vengono marcati con l'intero corrispondente alla profondità attuale ($ID = \text{profondità} + 2$). In fase di risalita la funzione ripristina la cella al valore 0. Questo meccanismo evita la creazione e la distruzione di oggetti a ogni livello dell'albero delle chiamate.

2 Compito uno: generazione dell'ambiente

Il primo compito consiste nella stesura di un generatore procedurale per creare la griglia e posizionarvi ostacoli non attraversabili. L'obiettivo è costruire gli ambienti per il collaudo dell'algoritmo di ricerca del cammino.

2.1 Personalizzazione della griglia tramite parametri

L'utente configura il generatore attraverso i parametri della riga di comando; la griglia prodotta viene poi salvata in un file JSON che ne registra dimensioni, tipologie di ostacoli e celle non attraversabili. I parametri di configurazione includono:

- **-rows** e **-cols**: numero di righe e colonne della griglia (dimensioni $R \times C$).
- **-seed**: seme numerico per il generatore pseudo-casuale, che assicura la totale riproducibilità degli ostacoli generati nelle successive campagne di prova.
- **-types**: elenco delle tipologie di ostacoli da inserire (con la possibilità di combinarle nello scenario **mix**).
- **-density**: frazione obiettivo di celle della griglia da occupare con ostacoli fisici (compresa tra 0.0 e 1.0).

2.2 Tipologie di ostacoli e modellizzazione logica

In conformità ai requisiti per i gruppi di tre studenti, l'applicativo modella cinque forme geometriche:

1. **Semplici (simple)**: celle singole non attraversabili sparse casualmente sulla griglia. Servono a valutare il comportamento di base dell'algoritmo in ambienti a densità uniforme, senza blocchi strutturati.
2. **Agglomerati (cluster)**: gruppi compatti di celle contigue generati simulando un cammino casuale a partire da celle "seme".

3. **Diagonali (diagonal)**: sequenze di due o tre celle adiacenti solo per uno spigolo. Questo scenario serve a valutare le regole sugli attraversamenti diagonali forzati.
4. **Recinti (enclosure)**: strutture perimetrali rettangolari dotate di area libera interna. Mettono alla prova l'algoritmo di potatura in caso di percorsi chiusi.
5. **Barre (bar)**: linee orizzontali o verticali munite di una singola apertura casuale. Costringono l'esplorazione a deviare attraverso un passaggio obbligato.

2.3 Algoritmo di generazione

La generazione della griglia è affidata alla classe `GridGenerator`. Per garantire l'accuratezza della densità complessiva obiettivo, il generatore inserisce prima le macrostrutture (barre, recinti, agglomerati, diagonali) e poi esegue un passo di riempimento fine tramite ostacoli semplici (`simple`) fino al raggiungimento esatto della quota prefissata.

Di seguito si riporta lo pseudocodice di generazione:

Algorithm 1 Generazione procedurale della griglia

```
1: procedure GENERAGRIGLIA(rows, cols, types, density, seed)
2:   grid  $\leftarrow$  NuovaGriglia(rows, cols)
3:   rng  $\leftarrow$  InizializzaGeneratoreCasuale(seed)
4:   total_cells  $\leftarrow$  rows  $\times$  cols
5:   target_obstacles  $\leftarrow$  Intero(total_cells  $\times$  density)
6:
7:    $\triangleright$  Fase 1: Generazione delle macrostrutture
8:   for each type  $\in$  types  $\setminus$  {"simple"} do
9:     local_seed  $\leftarrow$  rng.GeneraIntero()
10:    if type = "cluster" then
11:      CostruisciCluster(grid, target_obstacles  $\times$  0.3, local_seed)
12:    else if type = "diagonal" then
13:      CostruisciDiagonali(grid, target_obstacles  $\times$  0.2, local_seed)
14:    else if type = "enclosure" then
15:      CostruisciRecinti(grid, target_obstacles  $\times$  0.2, local_seed)
16:    else if type = "bar" then
17:      CostruisciBarre(grid, target_obstacles  $\times$  0.3, local_seed)
18:    end if
19:  end for
20:   $\triangleright$  Fase 2: Riempimento fine fino alla densità obiettivo
21:  current_density  $\leftarrow$  ConteggioOstacoli(grid)/total_cells
22:  if "simple"  $\in$  types or current_density < density then
23:    remaining_density  $\leftarrow$  max(0.01, density - current_density)
24:    local_seed  $\leftarrow$  rng.GeneraIntero()
25:    CostruisciSemplici(grid, remaining_density, local_seed)
26:  end if
27:  return grid
28: end procedure
```

3 Compito due: raggiungibilità e distanza libera

Il secondo compito si concentra sull'analisi topologica dello spazio locale a partire da una cella di origine O . Vengono introdotti i concetti geometrici di distanza libera (d_{lib}), cammino libero di tipo 1 e tipo 2, contesto, complemento e frontiera della chiusura.

3.1 Distanza libera (d_{lib}) e formulazione matematica

La distanza libera (d_{lib}) rappresenta il costo minimo del percorso ottimale tra due celle in una griglia con adiacenza a otto direzioni, assumendo l'assoluta assenza di ostacoli fisici interposti. Siano O e D le celle di origine e destinazione, e siano $\Delta x = |O.x - D.x|$ e $\Delta y = |O.y - D.y|$ le differenze assolute delle loro coordinate cartesiane sulla griglia.

Definiamo $\Delta_{\min} = \min(\Delta x, \Delta y)$ come il numero massimo di passi diagonali puri eseguibili e $\Delta_{\max} = \max(\Delta x, \Delta y)$ come la dimensione dominante dello spostamento. La formula matematica esatta per il calcolo di $d_{\text{lib}}(O, D)$ è:

$$d_{\text{lib}}(O, D) = \sqrt{2}\Delta_{\min} + (\Delta_{\max} - \Delta_{\min}) \quad (1)$$

Questa formula di costo $O(1)$ fornisce il limite inferiore richiesto dalla potatura geometrica descritta di seguito.

3.2 Contesto e complemento

La raggiungibilità locale tramite cammini liberi definisce due insiemi disgiunti di celle all'interno dello spazio libero:

- **Contesto:** l'insieme delle celle raggiungibili da O tramite un cammino libero di **tipo 1**, definito dalla sequenza prima mosse diagonali oblique, poi mosse cardinali residue, oppure da cammini composti esclusivamente da un tipo di mossa.

- **Complemento:** l'insieme delle celle raggiungibili da O **solo** tramite un cammino libero di **tipo 2**, definito dalla sequenza inversa prima mosse cardinali, poi mosse diagonali residue. Le celle che ammettono entrambi i tipi di cammino appartengono per convenzione al solo contesto.

L'unione di contesto e complemento forma la **chiusura** di O .

3.3 Proiezione di raggi geometrici ($O(\max(R, C)^2)$)

Per determinare gli insiemi contesto e complemento, il programma implementa la **proiezione di raggi**. L'analisi controlla le linee di raggiungibilità procedendo a raggiera a partire dall'origine, riducendo la complessità da quadratica ($O(R^2 \cdot C^2)$ per una ricerca cella per cella) a $O(\max(R, C)^2)$.

L'algoritmo per il **contesto (tipo 1)** opera come segue:

1. **Proiezione cardinale:** vengono tracciati i 4 raggi cardinali puri (Nord, Sud, Est, Ovest) a partire dall'origine O , arrestandosi non appena si incontra una cella ostacolo.
2. **Proiezione diagonale ed espansione a gomito:** per ciascuno dei 4 quadranti del piano cartesiano locale:
 - si proietta il raggio diagonale puro (NE, NO, SE, SO);
 - per ogni cella attraversabile raggiunta lungo questo raggio diagonale, si avvia un'espansione cardinale rettilinea parallela agli assi (espansione a gomito), che procede verso l'esterno fino a incontrare un ostacolo o il bordo della griglia.

L'algoritmo per il **complemento (tipo 2)** inverte simmetricamente le fasi di proiezione: per ciascuno dei 4 quadranti si proiettano prima i raggi cardinali e, da ogni cella libera raggiunta su di essi, si espandono raggi in diagonale pura. Al termine si escludono le celle già presenti nel contesto.

Questo accorgimento analizza ogni cella limitando le operazioni di lettura sulla matrice.

Di seguito si riporta lo pseudocodice dei due algoritmi. La scrittura $P + c$ indica la cella ottenuta spostando P di un passo lungo la direzione c ; una cella si dice *libera* se è interna alla griglia e non appartiene a *osta*.

Algorithm 2 Calcolo del contesto (celle raggiungibili con cammini liberi di tipo 1)

```

1: procedure CALCOLACONTESTO( $O, dim, osta$ )
2:    $contesto \leftarrow \{O\}$ 
3:   for each direzione cardinale  $c \in \{\text{Nord, Sud, Est, Ovest}\}$  do
4:      $P \leftarrow O + c$ 
5:     while  $P$  è libera do
6:        $contesto \leftarrow contesto \cup \{P\}$ 
7:        $P \leftarrow P + c$ 
8:     end while
9:   end for
10:  for each direzione diagonale  $d \in \{\text{NE, NO, SE, SO}\}$  do
11:     $P \leftarrow O + d$ 
12:    while  $P$  è libera do
13:       $contesto \leftarrow contesto \cup \{P\}$ 
14:       $\triangleright$  Espansione cardinale orizzontale e verticale a partire da  $P$ 
15:      for each  $c \in \{\text{componente orizzontale di } d, \text{componente verticale di } d\}$ 
16:        do
17:           $Q \leftarrow P + c$ 
18:          while  $Q$  è libera do
19:             $contesto \leftarrow contesto \cup \{Q\}$ 
20:             $Q \leftarrow Q + c$ 
21:          end while
22:        end for
23:       $P \leftarrow P + d$ 
24:    end while
25:  end for
26:  return  $contesto$ 
27: end procedure

```

Algorithm 3 Calcolo del complemento (celle raggiungibili solo con cammini liberi di tipo 2)

```

1: procedure CALCOLACOMPLEMENTO( $O, dim, osta, contesto$ )
2:    $complemento \leftarrow \emptyset$ 
3:   for each direzione diagonale  $d \in \{NE, NO, SE, SO\}$  do
4:     for each  $c \in \{\text{componente orizzontale di } d, \text{componente verticale di } d\}$ 
       do
5:        $P \leftarrow O + c$ 
6:       while  $P$  è libera do
7:          $\triangleright$  Espansione diagonale a partire dalla cella  $P$  del raggio cardinale
8:          $Q \leftarrow P + d$ 
9:         while  $Q$  è libera do
10:          if  $Q \notin contesto$  then
11:             $complemento \leftarrow complemento \cup \{Q\}$ 
12:          end if
13:           $Q \leftarrow Q + d$ 
14:        end while
15:         $P \leftarrow P + c$ 
16:      end while
17:    end for
18:  end for
19:  return  $complemento$ 
20: end procedure

```

3.4 Frontiera della chiusura

La **frontiera** della chiusura di O è definita formalmente come il sottoinsieme delle celle della chiusura che confinano (secondo l'intorno a otto direzioni) con almeno una cella attraversabile che si trova all'esterno della chiusura stessa:

$$Frontiera(O) = \{c \in Chiusura(O) \mid \exists v \in Vicini(c) \text{ t.c. } Stato(v) = 0 \wedge v \notin Chiusura(O)\} \quad (2)$$

Il codice estrae la frontiera limitando l'ispezione al vicinato delle sole celle incluse nella chiusura. Ad ogni cella di frontiera viene associato il tipo del cammino libero

da cui proviene (1 per il contesto, 2 per il complemento).

4 Compito tre: ricerca del cammino minimo

Il terzo compito descrive la progettazione e l'implementazione dell'algoritmo ricorsivo CAMMINOMIN per la ricerca del cammino minimo globale tra una cella di origine O e una cella di destinazione D . L'algoritmo opera su un grafo implicito i cui nodi sono costituiti dalle celle della griglia e i cui archi rappresentano i cammini liberi.

4.1 Definizione ricorsiva e flusso esecutivo

L'algoritmo si basa sulla scomposizione del percorso ottimale in una successione di sotto-cammini liberi. A ogni livello di ricorsione:

1. **Verifica del caso base:** si calcola la chiusura dell'origine corrente. Se la destinazione D appartiene a tale chiusura (ossia è raggiungibile con un singolo cammino libero di tipo 1 o tipo 2), la ricorsione si interrompe immediatamente ritornando la distanza ideale $d_{\text{lib}}(O, D)$ e i landmark associati.
2. **Espansione della frontiera:** se D non è direttamente raggiungibile, si estrae la frontiera locale della chiusura. Se la frontiera è vuota, il ramo corrente rappresenta un vicolo cieco e si ritorna un costo infinito.
3. **Esplorazione e ritracciamento:** per ciascun candidato di frontiera F si calcola il costo parziale $l_F = d_{\text{lib}}(O, F)$. Se la mossa supera le condizioni di potatura, si effettua una chiamata ricorsiva con origine in F e destinazione D , aggiornando dinamicamente lo stato della griglia.

Le chiusure calcolate nei livelli superiori dell'albero di ricorsione vengono marcate come ostacoli temporanei, per prevenire cicli e per evitare di riesplorare aree già visitate.

4.2 Analisi della potatura: riga 16 contro riga 17

Per ridurre lo spazio di ricerca e per evitare l'esplorazione di rami non ottimali, la specifica introduce due condizioni di potatura alternative:

- **Potatura debole (riga 16):** la cella F viene esplorata solo se la distanza ideale dal nodo corrente è inferiore al cammino ottimale globale provvisorio trovato finora:

$$l_F < \text{lunghezzaMin} \quad (3)$$

- **Potatura forte (riga 17):** sfrutta la stima ammissibile della distanza libera residua da F alla destinazione D , potando il ramo se la somma del cammino parziale e della stima minima rimanente non può battere l'ottimo corrente:

$$l_F + d_{\text{lib}}(F, D) < \text{lunghezzaMin} \quad (4)$$

La potatura forte è più onerosa per via del calcolo aggiuntivo di d_{lib} , ma riduce i nodi visitati in presenza di ostacoli complessi.

4.3 Potatura branch-and-bound globale

Lo pseudocodice valuta la condizione di riga 16/17 rispetto a `lunghezzaMin`, cioè il minimo *locale* della singola invocazione. L'implementazione la rafforza confrontandola con un **limite superiore globale** L_{glob} , condiviso da tutte le invocazioni ricorsive e aggiornato ogni volta che si trova un cammino completo più breve. Un candidato di frontiera F viene quindi esplorato solo se

$$\ell_{\text{acc}} + l_F + h(F) < L_{\text{glob}} \quad (5)$$

dove ℓ_{acc} è la lunghezza già accumulata dall'origine principale e $h(F)$ vale 0 nella potatura debole (riga 16) e $d_{\text{lib}}(F, D)$ in quella forte (riga 17). Poiché d_{lib} non sovrastima mai la distanza reale, la stima è ammissibile e la potatura non scarta mai un cammino ottimo: si tratta di una potatura branch-and-bound. Finché non si è trovato il primo cammino completo, L_{glob} vale $+\infty$ e non si pota nulla. È l'unica ottimizzazione di ricerca aggiunta rispetto allo pseudocodice e non richiede alcuna passata preliminare: il limite emerge dalla ricerca stessa.

4.4 Ottimizzazioni fisiche e implementative

L'implementazione prevede anche le seguenti ottimizzazioni strutturali:

- **Ordinamento euristico della frontiera:** prima di scorrere il ciclo sulle celle della frontiera, queste vengono ordinate per valori crescenti della distanza libera dalla destinazione. Questa strategia esplora per primi i percorsi più promettenti, riducendo il numero di rami da visitare prima di aggiornare il minimo globale.
- **Ritracciamento sul posto:** realizzato marcando lo stato della matrice condivisa con l'identificativo numerico della profondità corrente ed eseguendo il ripristino in tempo costante durante la risalita, senza allocazione di memoria.
- **Assenza di memorizzazione dei sotto-problemi:** un tentativo iniziale di introdurre la memoizzazione (con chiave data da origine, destinazione e insieme delle celle temporaneamente bloccate) ha rivelato un difetto logico. Il risultato di ogni sotto-problema dipende, attraverso la potatura globale, dalla lunghezza accumulata delle chiamate esterne, che non fa parte della chiave. Un risultato memorizzato con una lunghezza accumulata elevata, e quindi con i rami ottimali potati, poteva essere reimpiegato erroneamente in una chiamata con lunghezza accumulata minore, producendo una risposta subottimale e violando la simmetria richiesta dalla specifica (diapositiva 64). La memoizzazione è stata pertanto rimossa; le prestazioni sono garantite dalla potatura branch-and-bound globale e dall'ordinamento euristico della frontiera.

4.5 Requisiti funzionali per gruppi di 3 persone

Il software prevede le seguenti funzionalità per i gruppi di 3 persone:

- **Ricostruzione del cammino fisico (`reconstruct_path`):** a partire dalla sequenza di landmark ritornata $\langle (O, 0), (F_1, t_1), \dots, (D, t_D) \rangle$, il sistema ricostruisce l'esatta sequenza continua di celle attraversabili (mosse cardinali

e diagonali) unendo i singoli tratti liberi di tipo 1 o tipo 2 e validandone l'attraversabilità geometrica.

- **Gestione dell'interruzione per superamento del tempo limite:** l'ispezione del tempo trascorso a ogni chiamata ricorsiva consente di arrestare la ricerca in modo sicuro se si supera il tempo limite programmato dall'utente, salvando e restituendo la sequenza di celle di lunghezza minore individuata fino a quel momento.

5 Compito quattro: sperimentazione e analisi

Il quarto compito valuta le prestazioni del sistema. La sezione presenta le analisi di complessità spaziale e temporale, seguite dai risultati empirici.

5.1 Analisi di complessità spaziale asintotica

La complessità spaziale (occupazione di memoria centrale) è dettata dalla pila delle chiamate ricorsive e dalle strutture dati locali allocate a ogni livello:

- **Pila di ricorsione:** sia l la profondità massima della ricorsione, corrispondente al numero di landmark intermedi. Nel caso peggiore l è limitato dal numero totale di celle della griglia, quindi $l \leq R \times C$. Lo spazio della pila è $O(l)$.
- **Strutture locali di chiusura e frontiera:** a ogni livello ricorsivo il calcolo della chiusura corrente occupa al più $O(R \times C)$ celle. Lo stesso vale per la lista delle celle di frontiera estratte.
- **Gestione dello stato della griglia:** molti approcci semplici duplicano l'intero stato della griglia a ogni chiamata per simulare i nuovi ostacoli temporanei, portando la complessità spaziale totale a $O(R \times C \cdot l)$.

Grazie all'ottimizzazione del **ritracciamento sul posto** su array NumPy, l'implementazione mantiene un'unica griglia condivisa. Le chiusure temporanee vengono marcate tramite sovrascritture. Lo spazio di memoria allocato è legato alle variabili locali della ricorsione, contenendo l'occupazione reale.

5.2 Analisi di complessità temporale asintotica

Analisi della complessità temporale:

- **Caso peggiore (griglie complesse e ostruite):** in assenza di potature l'algoritmo esplora combinatoriamente i rami di frontiera con complessità $O(K^l)$, dove K è la dimensione della frontiera. La potatura forte e l'ordinamento euristico riducono il fattore di ramificazione; come mostrano i dati sperimentali,

sulle istanze provate il loro effetto è netto, pur non cambiando l'andamento asintotico.

- **Caso migliore (destinazione nella chiusura iniziale):** se la destinazione D appartiene al contesto o al complemento dell'origine O , l'algoritmo si arresta immediatamente al livello ricorsivo 0 (caso base). Il tempo di esecuzione è interamente dominato dal calcolo della chiusura iniziale tramite proiezione di raggi, con una complessità pari a:

$$T_{\text{best}} = O(\max(R, C)^2) \quad (6)$$

Questo si traduce in tempi di elaborazione inferiori al millisecondo anche per griglie di lato elevato (su lato 10 il calcolo richiede circa 0,3 ms).

5.3 Nota metodologica sulla misura

I tempi riportati nei grafici sono cronometrati in esecuzioni prive di profilatura della memoria. La misura del picco di memoria, infatti, introduce un sovraccarico di circa cinque volte: mantenerla attiva durante il cronometraggio falserebbe i tempi (sulla griglia 50×50 la stessa esecuzione passava da circa 1,7 s a circa 9,6 s). L'occupazione di memoria viene perciò rilevata in una passata separata e distinta da quella temporale.

5.4 Analisi dei risultati sperimentali e discussione dei grafici

La campagna sperimentale è stata eseguita su griglie da 10×10 fino a 200×200 , variando la densità degli ostacoli e analizzando le diverse strategie. Per le campagne su densità, potatura e ordinamento, ogni configurazione è stata provata su più coppie origine–destinazione (la coppia d'angolo, geometricamente più impegnativa, e quattro coppie casuali di celle attraversabili); le metriche riportate sono le mediane dei campioni raccolti. Come richiesto dalla specifica (diapositiva 64), ciascuna coppia considerata è stata sottoposta a doppia invocazione con i parametri

scambiati ($O \rightarrow D$ e $D \rightarrow O$): l'esito complessivo della verifica è registrato nel file `simmetria_campagna.json`. I risultati sono riassunti in sei grafici, discussi di seguito.

5.4.1 1. Tempo in funzione della densità di ostacoli

Il file `1_time_vs_density.png` illustra l'andamento del tempo di esecuzione su una griglia 50×50 al variare della densità degli ostacoli da 0.05 a 0.45. Il tempo mediano resta trascurabile (sotto i 10 ms) fino a una densità del 35%, poi sale ripidamente a circa 1,4 s al 45%. Questo è il fianco crescente della **transizione di fase** tipica dei problemi di ricerca:

- a densità basse lo spazio è aperto e la destinazione ricade spesso nella chiusura iniziale dell'origine, per cui il cammino viene trovato quasi istantaneamente con pochi landmark;
- avvicinandosi alla soglia critica lo spazio si frammenta: restano cammini tortuosi ma difficili da isolare, e l'algoritmo deve esplorare rami profondi prima di convergere, da cui la ripida crescita osservata al 45%;
- oltre la soglia di connettività (densità non incluse nelle prove) ci si attenderebbe un nuovo calo, perché molte coppie diventano irraggiungibili e l'algoritmo lo rileva subito potando l'intera frontiera.

5.4.2 2. Confronto delle configurazioni (potatura e ordinamento)

Il diagramma a barre in `2_pruning_comparison_boxplot.png` (in scala logaritmica) confronta i quattro scenari di esecuzione (potatura debole, potatura forte, ordinamento euristico e ordinamento casuale) sulla coppia d'angolo di una griglia 50×50 , per i diversi tipi di ostacolo. Entrambe le scelte risultano decisive:

- la **potatura forte (riga 17)** risolve ogni scenario in pochi millisecondi (da 0,006 a 0,014 s), mentre la **potatura debole (riga 16)** raggiunge il tempo limite di 20 s su quasi tutti i tipi (fa eccezione il tipo `bar`, circa 0,5 s). La

stima ammissibile $d_{\text{lib}}(F, D)$ aggiunta dalla riga 17 sfolta l'albero di ricerca in misura molto maggiore;

- l'**ordinamento euristico della frontiera** (per d_{lib} crescente dalla destinazione) è altrettanto importante: con l'ordinamento casuale la ricerca raggiunge il tempo limite su quattro tipi su cinque, mentre con quello euristico si conclude in millisecondi. Esplorare per primi i candidati più vicini alla destinazione consente di trovare presto un buon limite superiore, di cui la potatura branch-and-bound beneficia subito.

5.4.3 3. Crescita temporale asintotica (scala bilogaritmica)

Il grafico in `3_complexity_scaling_loglog.png` riporta i tempi sulla coppia d'angolo al crescere del lato (fino a 200×200), per la potatura debole e per quella forte. Si osserva che:

1. la **potatura forte** scala molto meglio: risolve fino al lato 50×50 in circa 1,6 s, mentre la **potatura debole** si conclude solo fino a 20×20 e raggiunge già il tempo limite (60 s) a 50×50 ;
2. da lato 100 in poi entrambe raggiungono il tempo limite, ma la potatura forte esplora meno nodi (a lato 100, circa 1,1 milioni di chiamate ricorsive contro i 2,0 milioni della debole);
3. il limite osservato è intrinseco all'algoritmo: la ricerca esaustiva del cammino ottimo fra due angoli opposti di una griglia densa è esponenziale nel numero di landmark, e da lato 100 in su nessuna configurazione la completa entro il tempo concesso.

In tutti i casi che superano il tempo limite l'algoritmo restituisce comunque il miglior cammino individuato fino a quel momento, segnalando che il calcolo non è stato completato (requisito funzionale della diapositiva 71).

5.4.4 4. Correlazione tra tempo e celle di frontiera

Il diagramma a dispersione in `4_scatter_time_vs_frontier.png` (scala bilogarithmica) mappa ogni prova ponendo sulle ascisse le celle di frontiera totali considerate e sulle ordinate il tempo di calcolo. I punti sono fortemente allineati: il tempo di esecuzione cresce in modo regolare con il volume di frontiera analizzato. La pendenza della retta di regressione è circa 0,82, leggermente sotto-lineare, coerente con un costo per singola cella di frontiera che non cresce con la dimensione del problema, grazie alla rappresentazione compatta su matrice NumPy.

5.4.5 5. Correlazione tra tempo e invocazioni ricorsive

Il grafico in `5_scatter_time_vs_recursion.png` mappa il tempo in funzione delle chiamate ricorsive totali, colorando i punti in base alla lunghezza del cammino. L'allineamento è marcato (pendenza di regressione circa 0,94): il numero di chiamate ricorsive è il fattore primario del costo temporale, a conferma del fatto che contenere la pila delle chiamate è la leva principale sulle prestazioni.

5.4.6 6. Verifica formale della simmetria

Il grafico a barre in `6_symmetry_per_type.png` mostra l'esito delle prove di simmetria ($O \rightarrow D$ contro $D \rightarrow O$) eseguite su cinque coppie casuali per ciascuna tipologia di ostacolo: tutte e cinque le coppie risultano verificate per ognuno dei cinque tipi, senza fallimenti né esclusioni. A queste prove dedicate si aggiunge la doppia invocazione effettuata su tutte le coppie delle campagne prestazionali (si veda l'introduzione di questa sezione): su 56 coppie totali, 49 sono risultate verificabili (le altre 7 escluse perché una delle due direzioni ha raggiunto il tempo limite) e nessuna ha prodotto incongruenze metriche. Si registra quindi il 100% di successi sulle coppie verificabili, a validazione sperimentale dell'esattezza dell'implementazione, come richiesto esplicitamente dalla specifica (diapositive 63–64).

6 Conclusioni

In questa relazione abbiamo documentato l'implementazione e la sperimentazione di un sistema per la ricerca del cammino a otto direzioni su griglie bidimensionali. Il software è stato sviluppato in Python utilizzando la libreria NumPy per la gestione vettoriale della griglia.

6.1 Limitazioni riscontrate

L'implementazione presenta le seguenti limitazioni, emerse dalle prove di esecuzione:

1. **Scalabilità della ricerca esaustiva:** su griglie con densità di ostacoli del 20%, le interrogazioni angolo-angolo si completano in pochi secondi fino al lato 50×50 (circa 1,7 s), ma da 100×100 in su la ricerca esaustiva del cammino ottimo supera il tempo limite (60 s) in tutte le configurazioni provate, richiedendo milioni di chiamate ricorsive. In questi casi l'algoritmo restituisce comunque il miglior cammino individuato, senza però dimostrarne l'ottimalità entro il tempo concesso. Le ottimizzazioni adottate (potatura forte e ordinamento euristico della frontiera) ne riducono le costanti, ma non l'andamento asintotico: poiché i risultati dei sotto-problemi non vengono memorizzati (la memoizzazione è stata rimossa per il difetto logico descritto nel Compito 3), sulle griglie molto ostruite le stesse chiusure intermedie vengono ricalcolate più volte.
2. **Limite di profondità della pila di ricorsione:** l'architettura ricorsiva fa affidamento sulla pila delle chiamate gestita dall'interprete. Sebbene in Python sia possibile innalzare questo limite tramite la chiamata `sys.setrecursionlimit(15000)`, per percorsi con migliaia di punti di riferimento intermedi il sistema rischia di incorrere in un errore di `RecursionError`.

6.2 Motivazioni delle scelte implementative

Le scelte di progettazione adottate mirano a contenere l'occupazione di memoria e a limitare il numero di nodi esplorati:

- **Gestione della memoria:** la rappresentazione della griglia come matrice NumPy di tipo `uint8` alloca un solo byte contiguo per cella. Unita al ritracciamento sul posto, che riusa l'unica matrice condivisa invece di duplicarla a ogni livello, questa scelta evita la proliferazione di oggetti tipica di un modello in cui ogni cella è un'istanza distinta.
- **Contenimento del tempo di calcolo:** la potatura branch-and-bound globale e l'ordinamento euristico della frontiera mirano a sfoltire l'albero di esplorazione. Sulle istanze risolubili entro il tempo limite questi accorgimenti riducono le costanti di calcolo; sulle griglie più grandi, però, non bastano a evitare il superamento del tempo limite (si veda il Compito 4).

6.3 Sviluppi futuri

Possibili sviluppi futuri includono:

- **Conversione in un algoritmo iterativo con pila esplicita:** la trasformazione della ricorsione in una struttura iterativa basata su una pila gestita esplicitamente in memoria dinamica eliminerebbe il legame con la pila delle chiamate dell'interprete. Questo consentirebbe di elaborare cammini di qualsiasi lunghezza o profondità senza il rischio di esaurire la pila.
- **Reintroduzione della memoizzazione con politica di sfratto (LRU/LFU):** si potrebbe riconsiderare una memoria dei sotto-problemi, risolvendo però il difetto logico che ne aveva imposto la rimozione, ossia la dipendenza del risultato dalla lunghezza accumulata delle chiamate esterne, non inclusa nella chiave. Per limitarne l'occupazione, tale memoria andrebbe dotata di capacità massima e di una politica di espulsione come *Least Recently Used* (LRU) o

Least Frequently Used (LFU), in modo da rimuovere automaticamente gli stati meno utilizzati una volta raggiunta la soglia.

7 Uso del codice

Questo capitolo fornisce il manuale utente completo e la documentazione tecnica per configurare, eseguire e validare il software sviluppato. Il punto di ingresso principale è l'interfaccia a riga di comando scritta in Python, che implementa tutti i compiti dell'elaborato.

7.1 Configurazione e requisiti di sistema

Il software è scritto in **Python 3.13**. Per eseguire l'applicazione e riprodurre i risultati sperimentali è necessario configurare l'ambiente virtuale installando le dipendenze dichiarate:

```
# Creazione dell'ambiente virtuale
python3 -m venv .venv

# Attivazione dell'ambiente
source .venv/bin/activate

# Installazione delle librerie necessarie (NumPy, Matplotlib)
pip install -r requirements.txt
```

7.2 Configurazione del file JSON delle griglie

Il file JSON utilizzato per memorizzare e caricare la struttura delle griglie adotta una sintassi strutturata, che dichiara esplicitamente le dimensioni della matrice, le tipologie di ostacoli con cui la griglia è stata generata (riportate poi nel riassunto dell'esecuzione, come richiesto dalla diapositiva 71) e le coordinate bidimensionali di ciascuna cella ostacolo fisso:

```
{
  "rows": 50,
  "cols": 50,
```

```

"types": ["simple", "cluster"],
"obstacles": [
    [0, 5],
    [0, 6],
    [1, 10],
    ...
]
}

```

7.3 Guida all'uso della riga di comando

Tutte le funzionalità del software sono esposte tramite il modulo principale `main.py`. I comandi devono essere eseguiti con l'interprete dell'ambiente virtuale e includendo la variabile d'ambiente `PYTHONPATH=.`

7.3.1 1. Generazione di griglie (Compito 1)

Per generare proceduralmente ostacoli misti (`simple`, `cluster`, `diagonal`, `enclosure`, `bar` o lo scenario combinato `mix`) con densità configurabile e salvare l'esito in formato JSON e in immagine PNG ad alta risoluzione:

```

PYTHONPATH=. .venv/bin/python main.py generate --rows 50 --cols 50 \
--types mix --density 0.2 --seed 42 \
-o grids/grid.json --save-img results/test_grid_generated.png

```

7.3.2 2. Calcolo di contesto, complemento e frontiera (Compito 2)

Per tracciare i raggi geometrici dall'origine e calcolare l'insieme delle celle appartenenti al contesto (tipo 1), al complemento (tipo 2) e alla frontiera locale:

```

PYTHONPATH=. .venv/bin/python main.py context --grid grids/grid.json \
--origin 10,10 --type both

```

7.3.3 3. Calcolo della distanza ideale d_{lib}

Per calcolare la distanza minima teorica priva di ostacoli su una griglia con adiacenza a otto direzioni tramite la formula geometrica chiusa:

```
PYTHONPATH=. .venv/bin/python main.py dlib --origin 10,10 --dest 40,40
```

7.3.4 4. Calcolo del cammino minimo (Compito 3)

Per calcolare il cammino minimo tramite ritracciamento ricorsivo basato su landmark (con la potatura branch-and-bound globale attiva), esportare l'immagine PNG ad alta risoluzione del percorso e stampare il riassunto JSON strutturato (diapositiva 71):

```
PYTHONPATH=. .venv/bin/python main.py camminomin --grid grids/grid.json \
--origin 0,0 --dest 49,49 --strong-pruning --summary \
--save-img results/test_path_resolved.png
```

L'inclusione del parametro `-strong-pruning` attiva la potatura tramite riga 17, mentre la sua assenza mantiene attiva la riga 16. Il parametro `-summary` stampa il resoconto strutturato con il conteggio delle celle di frontiera analizzate, il numero di potature eseguite e l'altezza massima raggiunta dalla pila ricorsiva.

7.3.5 5. Campagna sperimentale completa (Compito 4)

Per lanciare l'intera batteria automatizzata di prove sperimentali su griglie scalabili (fino a 200×200), densità variabili e strategie di potatura, esportando i sei grafici analitici discussi nel capitolo precedente nella cartella dei risultati:

```
PYTHONPATH=. .venv/bin/python main.py experiment --output-dir results
```

7.3.6 6. Verifica di simmetria formale

Per lanciare la verifica automatica di simmetria ($O \leftrightarrow D$ contro $D \leftrightarrow O$) su un numero N di coppie casuali di nodi:

```
PYTHONPATH=. .venv/bin/python main.py experiment --verify-symmetry-count 10 \
--symmetry-grid-size 20
```

7.4 Esecuzione della batteria di prove unitarie

Il progetto include una copertura completa di prove unitarie automatiche, che validano l'esattezza di tutti i moduli geometrici e logici (distanza libera, proiezione di raggi, compattazione, ritracciamento sul posto, salvataggio e caricamento della griglia). Per avviare le prove unitarie:

```
PYTHONPATH=. .venv/bin/python -m unittest discover -s tests -p "test_*.py"
```